Aplicación de los pilares de la POO en el proyecto EmpresaMotos

Autor: Camilo Tarazona Orozco · **Materia:** Lógica de Programación 2 · **Lenguaje:** Java (NetBeans, Maven)

1. Descripción general del proyecto

EmpresaMotos es un sistema en Java que administra una empresa de venta de motos mediante un menú por consola. Permite registrar motos deportivas y urbanas, mostrar el inventario, buscar una moto por modelo, registrar clientes y realizar ventas con su factura. El código está organizado en cuatro paquetes: `modelo` (Moto, MotoDeportiva, MotoUrbana, Cliente, Vendedor y Venta), `excepciones` (las tres excepciones personalizadas), `servicio` (EmpresaMotos, que guarda el inventario y los clientes) y el paquete raíz con la clase `Main`, que contiene el menú.

En este documento se explica cómo se aplican en el proyecto los cuatro pilares de la programación orientada a objetos: abstracción, encapsulamiento, herencia y polimorfismo.

2. Abstracción

La abstracción consiste en representar solo las características esenciales de un objeto y ocultar los detalles que no son necesarios. En el proyecto se aplica de la siguiente manera:

- La clase `Moto` es **abstracta** (`public abstract class Moto`). Representa el concepto general de una moto con los datos que comparten todas: marca, modelo, precio y disponibilidad. Como es abstracta, no se pueden crear objetos de tipo `Moto` directamente: siempre hay que crear una moto de un tipo concreto.
- `Moto` declara el método abstracto `public abstract double calcularPrecioFinal();`, que no tiene cuerpo. Con esto se define **qué** debe hacer toda moto (calcular su precio final) sin decir **cómo**, y se obliga a cada clase hija a implementarlo.
- La clase `Venta` también abstrae el proceso de venta: quien la usa solo llama a `realizarVenta()` y `generarFactura()`, sin conocer las validaciones y los cálculos internos.

Dónde se ve: `Moto.java` (clase abstracta y método abstracto) y `Venta.java`.

3. Encapsulamiento

El encapsulamiento protege los datos de un objeto, ocultándolos y permitiendo el acceso solo mediante métodos controlados.

- En todas las clases del modelo los atributos son **privados** (por ejemplo, `private String marca;` y `private double precio;` en `Moto`). En el diagrama de clases se indican con el símbolo `-`.
- El acceso se hace mediante **getters y setters** públicos, como `getPrecio()` y `setPrecio(double precio)`.
- Los setters pueden **validar** los datos. En `setPrecio` se comprueba que el precio sea mayor que cero y, si no lo es, se lanza `PrecioInvalidoException`. El constructor de `Moto` reutiliza ese mismo setter, por lo que es imposible crear una moto con un precio inválido.
- Algunos atributos tienen acceso restringido a propósito. En `Venta` no existen setters: el atributo `total` solo cambia al ejecutar `calcularTotal()`, y la disponibilidad de la moto cambia únicamente cuando `realizarVenta()` completa la venta.
- La lista `inventario` de `EmpresaMotos` es privada. Las motos solo se agregan con `registrarMoto()` y se consultan con `buscarMoto()` o `mostrarInventario()`.

Dónde se ve: `Moto.java` (atributos privados y validación en `setPrecio`), y las clases `Cliente`, `Vendedor`, `Venta` y `EmpresaMotos`.

4. Herencia

La herencia permite que una clase reutilice los atributos y métodos de otra clase y le agregue características propias.

- `MotoDeportiva` y `MotoUrbana` **heredan** de `Moto` con la palabra clave `extends`. Reciben automáticamente marca, modelo, precio, disponibilidad y todos sus métodos, sin escribirlos de nuevo.
- Cada hija agrega solo lo que la distingue: `MotoDeportiva` agrega `cilindrada` (int) y `MotoUrbana` agrega `consumo` (double, en km por galón).
- Los constructores de las hijas llaman al constructor de la clase padre con `super(marca, modelo, precio)`, que a su vez ejecuta la validación del precio.
- Gracias a la herencia, la lista `List<Moto> inventario` de `EmpresaMotos` puede guardar motos deportivas y urbanas juntas, porque ambas son un tipo de `Moto`.

Dónde se ve: `MotoDeportiva.java` y `MotoUrbana.java` (`extends Moto` y `super(...)`).

Las tres excepciones personalizadas también usan herencia: `PrecioInvalidoException`, `MotoNoDisponibleException` y `ClienteNoRegistradoException` extienden de la clase `Exception` de Java.

5. Polimorfismo

El polimorfismo permite que un mismo método tenga comportamientos distintos según el tipo real del objeto que lo ejecuta.

- `calcularPrecioFinal()` está declarado en `Moto` y **se sobrescribe** (`@Override`) en cada hija con una lógica diferente:
  - En `MotoDeportiva` se aplica un recargo del 15 % si la cilindrada es mayor a 600 cc y del 10 % en los demás casos.
  - En `MotoUrbana` se aplica un recargo fijo del 5 %.
- `mostrarInformacion()` también se sobrescribe en las hijas: llama a `super.mostrarInformacion()` para mostrar los datos comunes y luego agrega los datos propios (cilindrada o consumo) y el precio final.
- El polimorfismo se aprovecha en dos lugares del sistema. En `Venta.calcularTotal()` se ejecuta `moto.calcularPrecioFinal()` sobre una variable de tipo `Moto`, sin importar si es deportiva o urbana. En `EmpresaMotos.mostrarInventario()` se recorre la lista de `Moto` y cada una se muestra a su manera.
- Ejemplo de la ejecución: una Yamaha R6 de 600 cc con precio base de $40.000.000 queda en $44.000.000 (recargo del 10 %), y una Honda CB125 de $7.000.000 queda en $7.350.000 (recargo del 5 %). Ambas se calculan con la misma llamada.

Dónde se ve: `MotoDeportiva.java`, `MotoUrbana.java`, `Venta.calcularTotal()` y `EmpresaMotos.mostrarInventario()`.

6. Resumen de los pilares en el código

Pilar	Dónde se aplica	Cómo se aplica
Abstracción	`Moto`	Clase abstracta con el método abstracto `calcularPrecioFinal()`

Pilar	Dónde se aplica	Cómo se aplica
Encapsulamiento	Todas las clases del modelo	Atributos privados, getters y setters, validación en `setPrecio`
Herencia	`MotoDeportiva`, `MotoUrbana`	`extends Moto` y `super(...)`; reutilizan los atributos y métodos del padre
Polimorfismo	`calcularPrecioFinal()`, `mostrarInformacion()`	Método sobrescrito en cada hija y llamado desde `Venta` y `EmpresaMotos`

7. Otros conceptos aplicados

- **Composición / asociación:** `EmpresaMotos` contiene la lista de motos y la lista de clientes. `Venta` se asocia con una `Moto`, un `Cliente` y un `Vendedor`. `Cliente` y `Vendedor` están asociados porque el cliente compra a través del vendedor.
- **Manejo de excepciones:** `PrecioInvalidoException` se lanza cuando se intenta registrar una moto con precio cero o negativo. `MotoNoDisponibleException` se lanza cuando se intenta vender una moto que ya está agotada. `ClienteNoRegistradoException` se lanza cuando se intenta realizar una venta a un cliente inexistente.
- **Organización en paquetes:** separar `modelo`, `excepciones` y `servicio` mantiene el código ordenado y facilita su mantenimiento.

8. Conclusión

El proyecto EmpresaMotos aplica los cuatro pilares de la POO de forma conjunta: la abstracción define el concepto general de moto, el encapsulamiento protege sus datos, la herencia permite crear los tipos de moto sin repetir código y el polimorfismo hace que cada tipo calcule su precio final de manera propia. Esto hace que el sistema sea ordenado, fácil de ampliar (por ejemplo, agregando un nuevo tipo de moto solo requiere una nueva clase hija) y más seguro frente a datos inválidos.